웹 기술 이론 4장

반응형 설계 이론

유동 레이아웃, 브레이크포인트 정하기, 이미지·타이포·여백의 반응형

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

학습 목표

① 반응형 웹의 세 축(유동 그리드·유동 이미지·미디어쿼리)을 설명할 수 있다.

② 브레이크포인트를 기기가 아니라 콘텐츠 기준으로 정할 수 있다.

③ 모바일 우선 설계의 이점을 설명하고 적용할 수 있다.

④ 여백과 컨테이너 폭을 변수 하나로 일관되게 관리할 수 있다.

⑤ 반응형이 깨지는 것을 검증하는 방법을 실행할 수 있다.

이론 4장

핵심 키워드

반응형 · 유동 그리드 · viewport · 브레이크포인트 · 모바일 우선 · clamp · 컨테이너 폭 · 검증

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

이론 4장 · 1

반응형 웹의 세 축

2010년 이선 마콧(Ethan Marcotte)이 "Responsive Web Design"이라는 글에서 제시한 세 가지 원칙이 지금도 그대로 통합니다.

반응형 웹의 세 축

1. **유동 그리드(fluid grid)**
   폭을 px이 아니라 비율(%·fr)로 잡는다.
2. **유동 이미지(flexible image)**
   이미지가 부모보다 커지지 않게 한다.
3. **미디어쿼리(media query)**
   특정 폭에서 배치 자체를 바꾼다.

반응형 웹의 세 축

1. 세 번째만 알고 앞의 둘을 빠뜨리면 브레이크포인트 사이 구간에서 계속 깨집니다.

2. 미디어쿼리는 "몇 지점에서 크게 바꾸는" 도구일 뿐, 그 사이를 메우는 것은 유동 레이아웃입니다.

이론 4장 · 2

반드시 있어야 하는 한 줄

viewport 메타 태그

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

반드시 있어야 하는 한 줄 (1/2)

1. 이 줄이 없으면 휴대폰 브라우저는 "이건 980px짜리 데스크톱 페이지겠지" 하고 통째로 축소해 보여 줍니다.
2. 그러면 CSS가 보는 뷰포트 폭이 980px이 되어 미디어쿼리가 아예 발동하지 않습니다.
3. 반응형 CSS를 아무리 잘 써도 소용이 없어집니다.

반드시 있어야 하는 한 줄 (2/2)

1. user-scalable=no 나 maximum-scale=1 은 쓰지 마세요.
2. 확대해서 봐야 하는 사람의 확대를 막는 것이라 접근성 위반입니다.
3. 요즘 iOS는 이 설정을 무시하기도 합니다.

이론 4장 · 3

브레이크포인트를 정하는 기준

흔히 "모바일 768px, 태블릿 1024px"처럼 외워 쓰지만, 이것은 기기 목록이 계속 바뀌기 때문에 좋은 기준이 아닙니다.

브레이크포인트를 정하는 기준

1. **좋은 기준은 하나뿐입니다 — 내 디자인이 깨지는 지점.**

브레이크포인트를 찾는 방법

정리

1. **브라우저 창을 천천히 줄여 봅니다.**
   어색해지는 순간이 있습니다 — 글줄이 너무 길어지거나, 카드가 찌그러지거나, 여백이 사라지는 지점.

2. **그 지점에 브레이크포인트를 넣습니다.**
   그 값이 733px 같은 어중간한 숫자여도 상관없습니다.

3. **개수는 적을수록 좋습니다.**
   브레이크포인트가 많을수록 유지보수가 기하급수로 어려워집니다. 보통 2~3개면 충분합니다.

4. **유동 레이아웃을 먼저 잘 만들면**
   브레이크포인트가 덜 필요해집니다.

브레이크포인트를 정하는 기준

참고용 관례	대상	보통 하는 일
~480px	작은 휴대폰	1열, 햄버거 메뉴
~768px	휴대폰·작은 태블릿	1~2열, 사이드바 접기
~1024px	태블릿·작은 노트북	2~3열
1025px~	데스크톱	전체 배치

이론 4장 · 4

모바일 우선(Mobile First)

두 방향

```
/* ① 데스크톱 우선 — max-width로 좁혀 감 */
.grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }
@media (max-width: 1024px) { .grid { grid-template-columns: repeat(2, minmax(0, 1fr)); } }
@media (max-width: 768px)  { .grid { grid-template-columns: 1fr; } }

/* ② 모바일 우선 — min-width로 넓혀 감 (권장) */
.grid { grid-template-columns: 1fr; }
@media (min-width: 769px)  { .grid { grid-template-columns: repeat(2, minmax(0, 1fr)); } }
@media (min-width: 1025px) { .grid { grid-template-columns: repeat(3, minmax(0, 1fr)); } }
```

모바일 우선을 권하는 이유

정리

1. **방문자 대부분이 휴대폰**
   입니다. 기본 스타일이 모바일이면 휴대폰은 미디어쿼리를 읽을 필요조차 없습니다.

2. **좁은 화면에서 무엇이 정말 필요한지 먼저 생각하게**
   됩니다. 넓은 화면부터 만들면 "모바일에서 뭘 빼지?"가 되어 결과가 나빠집니다.

3. **CSS가 단순해집니다.**
   덮어쓰기가 아니라 더하기가 되기 때문입니다.

모바일 우선(Mobile First)

1. 미디어쿼리는 좁은 것부터 넓은 것 순서로 씁니다.

2. 순서가 뒤바뀌면 뒤에 온 규칙이 앞을 덮어 예상과 다르게 동작합니다.

이론 4장 · 5

여백과 컨테이너 폭

섹션마다 좌우 여백을 하드코딩하면 새 섹션을 추가할 때 조용히 빠뜨립니다.

여백과 컨테이너 폭

1. 실제로 한 사이트에서 영역별 여백이 12/16/20/28px로 제각각인 채 오래 방치된 사례가 있습니다.
2. 데스크톱만 맞아서 눈에 안 띄었던 것입니다.

변수 하나로 모으기

```css
:root {
  --container-wide:  1280px;   /* 값은 "내용 폭" — 패딩 제외 */
  --container-read:  1100px;   /* 항목이 긴 목록 */
  --container-prose:  860px;   /* 산문 페이지 */
  --pad: 20px;
}
@media (min-width: 640px)  { :root { --pad: 28px; } }
@media (min-width: 1024px) { :root { --pad: 40px; } }

.inner {
  max-width: calc(var(--container-wide) + 2 * var(--pad));
  padding: 0 var(--pad);
  margin: 0 auto;
}
```

이 방식이 지키는 것

정리

1. **여백을 한 곳에서 바꾼다.**
   새 섹션을 추가해도 자동으로 같은 여백이 적용됩니다.

2. **max-width 에 패딩을 포함시키지 않는다.**
   border-box에서 패딩이 바뀌면 내용 폭까지 흔들리기 때문입니다.

3. **장식 요소도 같은 변수에 묶는다.**
   right: 14px 로 고정하면 본문 여백이 40px일 때 혼자 어긋납니다 → right: calc(var(--pad) + 4px).

4. **전면 배경이 필요하면 2겹으로.**
   섹션(배경 전면) > inner(폭 제한) 구조를 쓰되, 모든 inner가 같은 변수를 쓰게 합니다.

여백과 컨테이너 폭

1. 패딩을 특정 폭에서 되레 줄이면 안 됩니다.
2. "화면이 넓어졌는데 여백이 줄어드는 역전"이 생깁니다.
3. 실제로 1359px→1360px 경계에서 여백이 79.5px→64px로 줄어든 사례가 있었습니다 — 스크린샷으로는 절대 못 잡습니다.

이론 4장 · 6

이미지와 미디어

기본형

```css
img, video {
  max-width: 100%;   /* 부모보다 커지지 않는다 */
  height: auto;      /* 비율 유지 */
  display: block;    /* 아래 생기는 4px 틈 제거 */
}
```

레이아웃이 덜컹거리지 않게

```
/* ① width/height 속성을 HTML에 지정 — 브라우저가 자리를 미리 잡는다 */
<img src="photo.jpg" alt="설명" width="800" height="600">

/* ② 또는 CSS에서 비율 고정 */
.thumb {
  aspect-ratio: 16 / 9;
  object-fit: cover;   /* 비율이 안 맞으면 잘라서 채운다 */
}
```

이미지와 미디어

1. 이미지가 로드되기 전에 높이가 0이었다가 갑자기 늘어나면 읽던 글이 아래로 밀립니다.
2. 이것을 레이아웃 이동(layout shift)이라고 하며, 사용자 경험 지표에서 큰 감점 요인입니다.

화면 크기에 따라 다른 이미지 주기

```
<img
  src="photo-800.jpg"
  srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1600.jpg 1600w"
  sizes="(max-width: 768px) 100vw, 50vw"
  alt="설명">

<!-- 브라우저가 화면 폭과 픽셀 밀도를 보고 알아서 고릅니다.
     휴대폰에 1600px 이미지를 내려보내지 않게 되어 로딩이 크게 빨라집니다. -->
```

이론 4장 · 7

타이포그래피의 반응형

clamp — 최소·유동·최대를 한 줄에

```css
h1 {
  font-size: clamp(1.75rem, 4vw + 1rem, 3rem);
  /*              최소값     유동값      최대값 */
}

/* 화면이 좁으면 1.75rem, 넓으면 3rem, 그 사이는 부드럽게 변합니다.
   미디어쿼리를 여러 개 쓰지 않아도 됩니다. */
```

읽기 좋은 글의 조건

정리

1. **본문은 16px 아래로 내리지 않습니다.**
   작은 글씨는 젊은 눈에도 피로합니다.
2. **한 줄에 45~75자.**
   너무 길면 다음 줄 시작점을 찾기 어렵습니다. 산문 페이지의 컨테이너를 860px 정도로 좁히는 이유입니다.
3. **줄 간격은 1.6~1.8.**
   한국어는 글자가 조밀해 영문보다 조금 넉넉한 편이 읽기 좋습니다.
4. **한국어 줄바꿈은 word-break: keep-all + overflow-wrap: break-word 한 세트.**
   좁은 화면일수록 이 차이가 크게 드러납니다.

이론 4장 · 8

반응형은 반드시 검증합니다

반응형은 눈으로 확인하지 않으면 깨진 줄도 모릅니다.

반응형은 반드시 검증합니다

1. **빌드는 통과하고 콘솔도 조용하기 때문입니다.**

검증 절차

정리

1. **개발자도구의 기기 도구 모음**
   (F12 → 휴대폰 아이콘)으로 360·390·430·768·1024·1280px를 차례로 봅니다.

2. **가로 스크롤이 생기는지**
   반드시 확인합니다. 생겼다면 어떤 요소가 넘치는지 Elements에서 하나씩 눌러 찾습니다.

3. **브라우저 창 크기를 직접 조절하는 것은 정확하지 않습니다.**
   OS 창을 바꿀 뿐이라 브라우저 크롬과 스케일 때문에 CSS 뷰포트와 어긋납니다(요청 500px에 실제 606px이 나오는 식). 기기 도구 모음을 쓰세요.

4. **여러 폭을 한 화면에 동시에 보려면**
   같은 페이지를 폭이 다른 iframe 여러 개에 띄웁니다. iframe 안의 미디어쿼리는 iframe 폭 기준으로 평가되므로 정확합니다.

반응형은 반드시 검증합니다

1. 고쳤다는 판정은 되돌려서 확인합니다.
2. 고친 뒤만 재면 원래 안 깨져 있었어도 통과합니다.
3. ① 고치고 잰다 ② 고치기 전 상태로 되돌려 다시 잰다 — 여기서 깨져야 판정이 사실이 됩니다 ③ 원복하고 전수 검증.
4. 이 절차 없이 "고쳤다"고 말하면 안 됩니다.

8.1 측정할 때 반복해 틀리는 것

잘못된 방법	왜 틀리는가	올바른 방법
window.innerWidth	스크롤바를 포함해 11~15px 커 보임	document.documentElement.clientWidth
숨김 요소를 계산에 포함	display: none 인 0×0 요소가 대칭 판정을 망침	크기 0인 요소를 제외
블록 요소의 바운딩 박스로 글자 위치 판단	h1 의 박스는 콘텐츠 폭 전체 — 글자는 훨씬 짧음	Range.getClientRects() 로 실제 글자 영역

8.1 측정할 때 반복해 틀리는 것

1. 한계도 알아 두세요.
2. iOS Safari의 주소창 높이 변화, safe-area(노치), 터치 관성 스크롤은 데스크톱 브라우저로 재현할 수 없습니다.
3. 그건 실제 기기로 확인해야 합니다.

웹 기술 이론 4장 마무리

4장 마무리

반응형 설계 이론 — 본문과 도해는 학습사이트에서 다시 읽습니다

- ✓ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시
- ✓ 함께 볼 주차: 6주차 · 7주차
- ✓ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다